

Hediyelik Eşyalar (Souvenirs)

Amaru yabancı bir dükkandan hediyelik eşya satın alıyor. Hediyelik eşyaların N tane **türü** vardır. Mağazada her türden sonsuz sayıda hediyelik eşya bulunmaktadır.

Her hediyelik eşya türünün sabit bir fiyatı vardır. Yani, i ($0 \leq i < N$) türündeki bir eşyanın fiyatı $P[i]$ madeni paradır. Burada $P[i]$ pozitif bir tam sayıdır.

Amaru, hediyelik eşya türlerinin fiyata göre azalan sırada sıralandığını ve hediyelik eşya fiyatlarının farklı olduğunu biliyor. Spesifik olarak, $P[0] > P[1] > \dots > P[N-1] > 0$. Ayrıca $P[0]$ değerini biliyor. Ancak bunların dışında, Amaru'nun hediyelik eşyaların fiyatları hakkında başka bir bilgisi maalesef bulunmuyor.

Amaru, hediyelik eşya satın almak için satıcıyla bir sürü işlem gerçekleştirecektir.

Her işlem aşağıdaki adımlardan oluşur:

1. Amaru satıcıya belirli miktarda (pozitif) madeni para verir.
2. Satıcı bu madeni paraları Amaru'nun göremeyeceği şekilde odanın arkasındaki masanın üzerine bir yığın halinde koyar.
3. Satıcı her bir hediyelik eşya türünü $0, 1, \dots, N-1$ sırasıyla tek tek değerlendirir. Her tür işlem başına **tam olarak bir kez** dikkate alınır.
 - Hediyelik eşya türü i göz önüne alındığında, eğer yığındaki mevcut madeni para sayısı en az $P[i]$ ise, o zaman
 - satıcı yığından $P[i]$ madeni parayı çıkarır ve
 - satıcı i türünde bir hediyelik eşyayı masaya koyar.
4. Satıcı, yığında kalan tüm madeni paraları ve masadaki tüm hediyelik eşyaları Amaru'ya verir.

Not: İşlem başlamadan önce masada madeni para veya hediyelik eşya olmadığına dikkat edin.

Göreviniz Amaru'ya aşağıdaki işlemleri gerçekleştirmesini söylemektir:

- her işlemde **en az bir** hediyelik eşya satın alır ve
 - genel olarak $0 \leq i < N$ olacak şekilde her i için **tam olarak** i adet hediyelik eşya satın alır.
- Not: Bu, Amaru'nun 0 türünde hiçbir hediyelik eşya satın almaması gerektiği anlamına geliyor.

Amaru'nun işlem sayısını minimize etmesine gerek yoktur ve sınırsız madeni para kaynağına sahiptir.

İmplementasyon Detayları

Aşağıdaki prosedürü kodlamalısınız.

```
void buy_souvenirs(int N, long long P0)
```

- N : hediyelik eşya türü sayısı
- $P0$: $P[0]$ 'ın değeri
- Her bir test case için bu prosedür tam olarak bir defa çağrılmalıdır.

Yukarıdaki prosedür, Amaru'ya bir işlem gerçekleştirmesi talimatını vermek için aşağıdaki prosedüre çağrılar yapabilir:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : Amaru'nun satıcıya verdiği madeni paraların sayısı.
- Prosedür bir çift döner. Çiftin ilk elemanı, satın alınan hediyelik eşya türlerini (artan sırada) içeren L dizisidir. İkinci eleman ise R tam sayısı olup, işlemden sonra Amaru'ya iade edilen madeni para sayısıdır.
- $P[0] > M \geq P[N - 1]$ olması gerekir. $P[0] > M$ koşulu, Amaru'nun 0 türünde hiçbir hediyelik eşya satın almamasını ve $M \geq P[N - 1]$ Amaru'nun en az bir hediyelik eşya satın almasını sağlar. Bu koşullar sağlanmadığı takdirde çözümünüz `Output isn't correct: Invalid argument` sonucunu alacaktır. Not: $P[0]$ 'ın aksine, $P[N - 1]$ değerinin girdide sağlanmadığına dikkat edin.
- Her bir test case'de prosedür en fazla 5000 kere çağrılabilir.

Graderin davranışı **uyarlanabilir (adaptif) değildir**. Bu, `buy_souvenirs` çağrılmadan önce P fiyat serisinin sabit olduğu anlamına gelir.

Kısıtlar

- $2 \leq N \leq 100$
- her bir i için $1 \leq P[i] \leq 10^{15}$ öyle ki $0 \leq i < N$.
- her bir i için $P[i] > P[i + 1]$ öyle ki $0 \leq i < N - 1$.

Altgörevler

Altgörev	Puan	Ek kısıtlar
1	4	$N = 2$
2	3	her bir i için $P[i] = N - i$ öyle ki $0 \leq i < N$.
3	14	her bir i için $P[i] \leq P[i + 1] + 2$ öyle ki $0 \leq i < N$.
4	18	$N = 3$
5	28	her bir i için $P[i + 1] + P[i + 2] \leq P[i]$ öyle ki $0 \leq i < N - 2$. her bir i için $P[i] \leq 2 \cdot P[i + 1]$ öyle ki $0 \leq i < N - 1$.
6	33	Ek kısıt yoktur.

Örnek

Aşağıdaki çağrıyı göz önüne alın.

```
buy_souvenirs(3, 4)
```

$N = 3$ tane hediyelik eşya türü vardır ve $P[0] = 4$. Sadece üç olası fiyat serisinin P olduğunu unutmayın: $[4, 3, 2]$, $[4, 3, 1]$, and $[4, 2, 1]$.

`buy_souvenirs` in `transaction(2)` çağırıldığını kabul edin. Çağrının $([2], 1)$ döndüğünü varsayalım; bu, Amaru'nun 2 türünde bir hediyelik eşya satın aldığı ve satıcının ona 1 madeni para geri verdiği anlamına gelir. Bu sayede $P = [4, 3, 1]$ sonucunu çıkarabileceğimizi gözlemleyelim:

- $P = [4, 3, 2]$ için, `transaction(2)`, $([2], 0)$ dönmüş olurdu.
- $P = [4, 2, 1]$ için, `transaction(2)`, $([1], 0)$ dönmüş olurdu.

Daha sonra `buy_souvenirs`, $([1], 0)$ değerini dönen `transaction(3)` çağırabilir; bu, Amaru'nun 1 türünde bir hediyelik eşya satın aldığı ve satıcının ona 0 adet madeni para geri verdiği anlamına gelir. Şimdiye kadar Amaru toplamda, 1 türünde bir hediyelik eşya ve 2 türünde bir hediyelik eşya satın almıştır.

Son olarak, `buy_souvenirs`, `transaction(1)` çağırabilir ve bu da $([2], 0)$ döner; bu da Amaru'nun 2 türünde bir hediyelik eşya satın aldığı anlamına gelir. Burada `transaction(2)` yi de kullanabileceğimizi unutmayın. Bu noktada, istenildiği gibi, Amaru'nun toplamda 1 türünde bir adet ve 2 türünde iki adet hediyelik eşyası bulunmaktadır.

Örnek Grader

Girdi formatı:

N

$P[0] \quad P[1] \quad \dots \quad P[N-1]$

Çıktı formatı:

$Q[0] \quad Q[1] \quad \dots \quad Q[N-1]$

Burada her i için olmak üzere $Q[i]$, i türünde satın alınan hediyelik eşyaların toplam sayısıdır, öyle ki $0 \leq i < N$.